SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: COMPUTER VISION with OpenCV
COURSE CODE: DSA02
G. Kushwanth(192424150)

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images. (BTL 3)

---

1. Grey Level Transformation

(a) Grey level transformation remaps every pixel intensity through a function s = T(r) so that the visual contrast and appearance of an image can be enhanced. It is used to brighten/darken images, expand contrast in a chosen range, or reverse intensities to highlight detail hidden in bright or dark regions, without changing the spatial arrangement of pixels.

(b) Negative transformation s = 255 − r:

```
r = 50  → s = 255 − 50  = 205
r = 100 → s = 255 − 100 = 155
r = 150 → s = 255 − 150 = 105
r = 200 → s = 255 − 200 = 55
```

2. Log Transformation

(a) Log transformation, s = c·log(1+r), compresses the higher end of the intensity range while stretching the lower (dark) end, because the log function grows quickly for small r and slowly for large r. This brings out details in dark regions of an image (e.g., Fourier spectra) that would otherwise be indistinguishable, at the cost of compressing bright details.

(b) s = 10·log10(1+r):

```
r = 0   → s = 10·log10(1)   = 10 × 0      = 0.000
r = 10  → s = 10·log10(11)  = 10 × 1.0414 = 10.414
r = 50  → s = 10·log10(51)  = 10 × 1.7076 = 17.076
r = 100 → s = 10·log10(101) = 10 × 2.0043 = 20.043
```

3. Histogram Processing

(a) Histogram equalization redistributes the intensity levels of an image so that the output histogram is as close to uniform as possible. This spreads out the most frequent intensity values, increasing the global contrast of the image, especially useful when the original histogram is concentrated in a narrow range.

(b) 3-bit image, total pixels N = 2+2+4+8 = 16. PDF and CDF:

```
r : freq : PDF (f/16) : CDF
0 :  2   : 0.125      : 0.125
1 :  2   : 0.125      : 0.250
2 :  4   : 0.250      : 0.500
3 :  8   : 0.500      : 1.000
```

Equalized level: s = round(CDF × (L−1)), L = 8 → L−1 = 7

```
s0 = round(0.125×7) = round(0.875) = 1
s1 = round(0.250×7) = round(1.75)  = 2
s2 = round(0.500×7) = round(3.5)   = 4
s3 = round(1.000×7) = round(7.0)   = 7
```

4. Spatial Filtering (Smoothing)

(a) Spatial smoothing filters replace each pixel with an average (or weighted average) of its neighbourhood. This blurs the image, suppressing high-frequency noise and small variations, and is typically used as a pre-processing step before edge detection or segmentation.

(b) 3×3 averaging filter on the region:

```
10 20 30
20 30 40
30 40 50
Sum = 10+20+30+20+30+40+30+40+50 = 270
New centre value = 270 / 9 = 30
```

5. Spatial Filtering (Sharpening)

(a) Sharpening filters emphasise rapid intensity changes (edges and fine detail) by responding to the second derivative of the image. Adding a Laplacian-type response back to the image boosts edges while leaving flat regions largely unaffected, making the image appear crisper.

(b) Laplacian mask [[0,-1,0],[-1,4,-1],[0,-1,0]] on:

```
10 10 10
10 20 10
10 10 10
Output = 4×(centre) − (sum of 4 neighbours)
        = 4×20 − (10+10+10+10) = 80 − 40 = 40
```

6. Frequency Domain Filtering

(a) Low-pass filtering in the frequency domain retains low-frequency components (which carry the bulk of image energy/smooth regions) and attenuates high frequencies, producing a smoothed image. High-pass filtering does the opposite — it removes/attenuates low frequencies and retains high frequencies, sharpening edges and fine detail while removing slowly varying background.

(b) F = [100, 80, 40, 20]; low-pass filter keeps only the first two (lowest-frequency) components:

7. Thresholding

(a) Global thresholding converts a grayscale image into a binary image by comparing every pixel to a single fixed threshold T: pixels with intensity ≥ T are set to 1 (object/foreground) and pixels below T are set to 0 (background). It is the simplest form of image segmentation.

(b) Pixel value = 200, T = 100. Since 200 ≥ 100:

8. Edge Detection

(a) Edge detection locates points in an image where intensity changes sharply, corresponding to object boundaries, texture changes, or discontinuities. It is typically performed by convolving the image with gradient operators (e.g. Sobel, Prewitt) and identifying pixels with large gradient magnitude.

(b) Horizontal Sobel mask [[-1,-2,-1],[0,0,0],[1,2,1]] on:

```
10 10 10
```

```
20 20 20
30 30 30
Gradient Gx = (-1×10 -2×10 -1×10) + (0) + (1×30 +2×30 +1×30)
            = -40 + 120 = 80
```

9. Region-Based Segmentation

(a) Region growing starts from one or more seed pixels and iteratively adds neighbouring pixels to the region as long as they satisfy a similarity criterion (e.g. |intensity − seed| ≤ threshold). The process stops when no more neighbouring pixels satisfy the criterion, producing a connected region of similar pixels.

(b) Seed = 50, threshold difference = 5 ⇒ acceptable range = [45, 55]. Example neighbourhood {48, 52, 60}:

```
|48-50| = 2 ≤ 5  → included in region
|52-50| = 2 ≤ 5  → included in region
|60-50| = 10 > 5 → excluded from region
```

10. Combined Enhancement & Segmentation

(a) Smoothing is applied before segmentation to suppress noise and small spurious intensity fluctuations. Without smoothing, noisy pixels can be mis-classified during thresholding/region growing, producing a fragmented or noisy segmentation mask; smoothing first gives a cleaner, more reliable boundary.

(b) Step 1 — Averaging: given mean = 48.

(c) (No separate part-c; combined with threshold step below.)

(d) Step 2 — Threshold at T = 50: since 48 < 50, the pixel is classified as background (0).

11. Power-Law (Gamma) Transformation

(a) Gamma correction, $s = c \cdot r^{\gamma}$, controls the overall brightness/contrast balance of an image. Values of $\gamma < 1$ expand the dark intensity range and compress the bright range (brightening shadows), while $\gamma > 1$ does the opposite (darkening the image, expanding bright detail). It is widely used to correct for the non-linear response of display/capture devices.

(b) With $\gamma = 0.5$ and $c = 1$, $s = r^{0.5} = \sqrt{r}$. Since square-root is concave, small r values are boosted proportionally more than large r values, so the transformation is justified when the image is generally dark and shadow detail needs to be brought out without overexposing bright areas.

(c) Apply $s = \sqrt{r}$ step by step:

```
r = 25  → s = √25  = 5
r = 100 → s = √100 = 10
r = 225 → s = √225 = 15
```

(d) Transformed values: s = [5, 10, 15]

(e) All three pixels become brighter in absolute terms, but the low value (25) is boosted by the largest relative factor (×0.2 of range vs ×0.067 originally), confirming that gamma < 1 preferentially enhances dark regions while compressing the increase given to already-bright pixels.

12. Contrast Stretching

(a) Contrast stretching is needed when an image's intensities occupy only a narrow band (here [50,150] out of the possible [0,255]), which makes the image look flat/low-contrast. Stretching this range to fill the full display range makes details easier to distinguish visually.

(b) Linear transform to map [50,150] → [0,255]: $s = (r - r_{min})/(r_{max} - r_{min}) \times 255 = (r - 50)/100 \times 255$. This is justified because it is the simplest transformation that is exact at both endpoints and linear (proportional) in between, preserving relative relationships between intensities.

(c) Apply step by step:

```
r = 50  → s = (50-50)/100  × 255 = 0
```

`r = 100 → s = (100-50)/100 × 255 = 127.5 ≈ 128`

`r = 150 → s = (150-50)/100 × 255 = 255`

(d) Output values: s = [0, 128, 255]

(e) The original 100-level spread [50,150] is now stretched to the full 255-level range, so intensity differences that were subtle before are now much more visually distinguishable — a clear contrast improvement.

13. Histogram Equalization (4-bit Image)

(a) Equalization is needed because the histogram is skewed (most pixels concentrated at levels 1 and 2), giving the image poor, uneven contrast. Redistributing intensities using the cumulative distribution spreads the histogram closer to uniform, improving contrast.

(b) The CDF is used because it is a monotonically non-decreasing function that naturally maps a skewed PDF onto an approximately uniform output distribution — scaling the CDF by the maximum output level (L−1) produces the new intensity value for each input level.

Total pixels N = 4+6+10+6 = 26. Using L = 16 gray levels for a 4-bit image (L−1 = 15).

(c) Cumulative distribution:

```
r : freq : PDF (f/26) : CDF
0 :  4   : 0.1538      : 0.1538
1 :  6   : 0.2308      : 0.3846
2 : 10   : 0.3846      : 0.7692
3 :  6   : 0.2308      : 1.0000
```

(d) New intensity levels: s = round(CDF × 15)

```
s0 = round(0.1538×15) = round(2.31)  = 2
s1 = round(0.3846×15) = round(5.77)  = 6
s2 = round(0.7692×15) = round(11.54) = 12
s3 = round(1.0000×15) = 15
```

(e) The four original levels {0,1,2,3}, which were bunched together, are spread out to {2,6,12,15} across the full 0–15 range — a clear expansion of contrast, especially separating the dominant level 2 (freq. 10) from its neighbours.

14. Median Filtering

(a) Salt-and-pepper noise appears as randomly scattered pixels forced to the extreme intensities (0 or 255/near-max), while the surrounding true image content is unaffected. It is an impulsive, non-Gaussian type of noise.

(b) The median filter is preferred over averaging because it replaces a pixel with the middle (rank-ordered) value of its neighbourhood rather than the mean; since impulsive noise values are extreme outliers, they get pushed to the ends of the sorted list and are excluded from the middle rank, so the filter removes salt-and-pepper noise without blurring edges the way averaging would.

(c) Apply 3×3 median filter to:

```
10 200 10
200 50 200
10 200 10
Sorted values: 10,10,10,10,50,200,200,200,200 (9 values)
Median (5th value) = 50
```

(d) Filtered center value = 50

(e) The filter output equals the true underlying value (50), completely rejecting the surrounding salt-and-pepper (10/200) noise — demonstrating the median filter's strong impulsive-noise removal capability.

15. Gaussian Smoothing

(a) The objective of smoothing here is to blur the image slightly to reduce noise and fine texture while preserving the overall structure/edges better than a plain average would.

(b) The Gaussian filter is preferred over a flat averaging filter because its weights fall off smoothly from the centre (following a Gaussian/bell curve, approximated here by the [1,2,1;2,4,2;1,2,1] mask), giving nearby pixels more influence than distant ones. This produces smoother, more natural blurring with fewer artefacts than uniform averaging.

(c) Apply the mask (kernel sum = 16) to:

```
10 20 10

20 40 20

10 20 10

Weighted sum = 1×10+2×20+1×10 + 2×20+4×40+2×20 + 1×10+2×20+1×10

             = 60 + 240 + 60 = 360

Normalized   = 360 / 16 = 22.5
```

(d) Center pixel (rounded) = 22 or 23 (typically taken as 23)

(e) The sharp central peak (40) is pulled down toward the surrounding background level, smoothing the local variation while the weighting still respects the centre pixel more than a flat average would.

16. High-Pass Filtering

(a) High-pass filtering is needed when fine detail and edges need to be emphasised, e.g. an image looks soft/blurred and boundaries need to stand out more clearly.

(b) The mask [[-1,-1,-1],[-1,8,-1],[-1,-1,-1]] is selected because its coefficients sum to zero, so flat/uniform regions produce a zero response (no change), while regions with a strong local difference (an edge) produce a large positive or negative response — exactly the behaviour needed to isolate and emphasise high-frequency detail.

Using the representative 3×3 region [[10,10,10],[10,20,10],[10,10,10]] (as in Q5) to illustrate the computation:

(c) Apply the mask:

```
Output = 8×(centre) − (sum of 8 neighbours)

       = 8×20 − (10×8) = 160 − 80 = 80
```

(d) Center value = 80

(e) The large positive response confirms a strong local edge at the centre pixel; this response would be added to (or replace) the original pixel to visually sharpen that region.

17. Frequency Domain High-Pass

(a) High frequencies in the Fourier domain correspond to sharp intensity transitions — edges, fine texture and noise. Preserving them keeps detail crisp, whereas discarding them (low-pass) would blur the image.

(b) High-pass filtering is justified when the goal is to sharpen an image or to isolate edges/fine structure, since it removes the slowly-varying background (low frequencies) and keeps the components responsible for detail.

(c) F = [10, 20, 80, 100]; remove the low-frequency components (keep the last two):

(d) Filtered output F' = [0, 0, 80, 100]

(e) Only the high-frequency components survive, so reconstructing the image from F' would emphasise edges and fine detail while removing smooth shading/background.

18. Adaptive Thresholding

(a) A single global threshold can fail when illumination varies across the image (e.g. shadows or uneven lighting), causing some regions to be mis-segmented even though a local threshold would classify them correctly.

(b) Adaptive thresholding is justified because it computes a threshold from local statistics (e.g. the local mean) for each neighbourhood, so it adapts to local brightness variations instead of relying on one global cut-off.

(c) Local threshold = local mean = 100. Pixels: [60, 80, 120, 140]

(d) Binary output: 60→0, 80→0, 120→1, 140→1, i.e. [0, 0, 1, 1]

(e) Pixels below the local mean are classed as background and those at/above it as foreground, correctly separating the two intensity groups present in this local neighbourhood.

19. Edge Detection (Vertical Sobel)

(a) The objective is to detect edges that run vertically, i.e. locate columns where intensity changes sharply from left to right.

(b) The vertical Sobel operator is chosen because its kernel has zero weight down the centre column and opposite-signed weights on the left/right columns (with extra weight on the middle row), making it selectively sensitive to horizontal intensity gradients (vertical edges) while suppressing vertical gradients.

(c) Apply mask [[-1,0,1],[-2,0,2],[-1,0,1]] to:

```
10 20 30
10 20 30
10 20 30
Gx = (-1×10+0+1×30) + (-2×10+0+2×30) + (-1×10+0+1×30)
   = 20 + 40 + 20 = 80
```

(d) Center value = 80

(e) The large gradient confirms a strong vertical edge, consistent with the intensity increasing steadily from left (10) to right (30) in every row.

20. Prewitt Edge Detection

(a) The Prewitt operator estimates the image gradient to detect edges, similar in purpose to Sobel but using uniform (unweighted) neighbour coefficients.

(b) Prewitt is sometimes used instead of Sobel when a simpler, computationally lighter operator is acceptable and slightly less noise suppression is tolerable; Sobel is generally preferred when better noise robustness (due to its centre-row/column weighting of 2) is needed, so the choice is a trade-off, not a strict improvement.

(c) Apply mask [[-1,0,1],[-1,0,1],[-1,0,1]] to:

```
10 20 30
10 20 30
10 20 30
Gx = (-10+30) + (-10+30) + (-10+30) = 20+20+20 = 60
```

(d) Output = 60

(e) A positive gradient of 60 again indicates a left-to-right vertical edge, slightly smaller in magnitude than the Sobel result (80) because Prewitt does not give extra weight to the centre row.

21. Region Splitting

(a) Region splitting deals with an image (or pixel set) that contains more than one distinct homogeneous group; a single description/threshold cannot represent the whole set well, so it must be divided into more uniform sub-regions.

(b) Splitting is justified when a region's variance/range exceeds an acceptable limit for one homogeneous class — here the pixel set clearly clusters into a low group and a high group, so splitting at a suitable threshold separates them correctly.

(c) Apply threshold = 80 to {40, 45, 100, 105}:

(d) Region A (< 80) = {40, 45}; Region B (≥ 80) = {100, 105}

(e) The two resulting regions are each internally homogeneous (values within 5 of each other), confirming that a single split at 80 cleanly separates the two underlying regions.

22. Region Merging

(a) Merging deals with the opposite problem to splitting: two adjacent regions that are actually part of the same object but were segmented separately, because over-segmentation produced regions that are too similar to justify remaining separate.

(b) Merging is justified when neighbouring regions' representative intensities differ by less than an acceptable tolerance, since such small differences are more likely due to noise/quantisation than a genuine object boundary.

(c) Regions [45,50] and [52,55], merge threshold = 5. Boundary values 50 and 52:

(d) $|52 - 50| = 2 \leq 5 \rightarrow$ merge condition satisfied

(e) Merged region = [45, 55]; since the two regions' intensities are close enough (difference well within the tolerance), they are combined into a single larger, homogeneous region.

23. Edge Linking

(a) Edge detectors often produce broken/discontinuous edge maps (gaps due to noise or weak gradients), which makes it hard to trace complete object boundaries directly.

(b) Edge linking (e.g. using local gradient direction/magnitude continuity, or morphological closing) is justified because it bridges small gaps between edge fragments that most likely belong to the same physical boundary, producing continuous contours.

(c) Apply linking to the sequence [1, 0, 1, 0, 1] (filling isolated single-pixel gaps between edge pixels):

(d) Linked result = [1, 1, 1, 1, 1]

(e) The two single-pixel gaps are closed, turning a broken edge sequence into one continuous edge — the typical goal of an edge-linking step before boundary/contour extraction.

24. Low-Pass Filtering (Frequency)

(a) Smoothing is required when noise or unwanted fine texture needs to be suppressed while preserving the overall (low-frequency) structure of the image.

(b) Low-pass filtering (LPF) is selected because it preserves the low-frequency components — which carry most of an image's energy/overall shape — while attenuating the high-frequency components responsible for noise and fine detail.

(c) F = [200, 150, 50, 20]; retain only the first two (low-frequency) components:

(d) Filtered output F' = [200, 150, 0, 0]

(e) Removing the high-frequency terms (50, 20) suppresses fine detail/noise while the dominant low-frequency content (200, 150) — and hence the overall image structure — is preserved.

25. Laplacian Sharpening (Variant)

(a) Sharpening is needed to counteract blur and emphasise edges/fine detail that may have been lost or softened during acquisition or prior smoothing.

(b) The mask [[0,-1,0],[-1,5,-1],[0,-1,0]] is justified because it is a standard Laplacian-based mask whose coefficients sum to 1 — it directly outputs an edge-enhanced image (original intensity plus the Laplacian response) in a single convolution, rather than requiring a separate addition step.

(c) Apply to [[10,10,10],[10,20,10],[10,10,10]]:

```
Output = 5×(centre) − (sum of 4 orthogonal neighbours)
       = 5×20 − (10+10+10+10) = 100 − 40 = 60
```

(d) Center value = 60

(e) Compared with the basic Laplacian mask in Q5 (output 40), this unit-sum variant gives a larger response (60) because it simultaneously enhances the edge and preserves/adds back the original pixel intensity.

26. Histogram Matching

(a) Histogram matching (specification) is needed when an image should be adjusted to resemble the intensity distribution of a reference image/histogram, rather than simply being equalized to a uniform distribution — useful for making multiple images visually consistent.

(b) The method is justified because it works by first equalizing both the input and the reference histograms (mapping each to a common uniform CDF), then composing the input-to-uniform mapping with the inverse of the reference-to-uniform mapping, so that input intensities are moved to whichever reference intensity has the closest matching cumulative probability.

(c) For input values [10, 50, 100], the matching procedure is:

```
1. Compute CDF_input(r) for each value.
2. Compute CDF_reference(z) for the reference histogram.
3. For each r, find z such that CDF_reference(z) is closest to CDF_input(r).
```

(d) New value for each input = that closest-matching reference intensity z.

(e) The result is an image whose histogram shape follows the reference distribution rather than a flat one, useful e.g. when matching exposure/contrast between two photographs of the same scene.

27. Noise Reduction Comparison

(a) This scenario again assumes impulsive salt-and-pepper noise: a minority of pixels are corrupted to extreme values while the rest of the image is clean.

(b) Averaging filters reduce noise by blending it into the neighbourhood, but because the mean is sensitive to outliers, a single extreme noisy pixel can noticeably shift the output; median filtering instead selects the middle ranked value, so extreme outliers are excluded entirely, giving much better noise suppression for impulsive noise (at the cost of not being ideal for Gaussian noise, where averaging performs relatively better).

(c) Apply both filters:

```
Average = (10+200+10+200+50+200+10+200+10)/9 = 900/9 = 100
Median  = middle of sorted {10,10,10,10,50,200,200,200,200} = 50
```

(d) Average output = 100 (heavily biased by noise); Median output = 50 (equals true value)

(e) The median filter output matches the true underlying pixel value exactly, while the average is dragged far above it by the surrounding noisy pixels — clearly demonstrating the median filter's superiority for salt-and-pepper noise.

28. Combined Filtering

(a) Smoothing followed by sharpening is often needed because smoothing alone removes noise but also blurs edges, so a subsequent sharpening step restores/enhances edge definition without reintroducing the original noise.

(b) The sequence (smooth, then sharpen) is justified rather than the reverse because sharpening a still-noisy image would amplify the noise (since sharpening responds to the second derivative, which is very sensitive to noise); smoothing first removes that noise so the later sharpening step enhances genuine edges only.

(c) Apply filters stepwise:

```
Step 1 (average, from Q4): centre → 30
Step 2 (Laplacian sharpen, from Q5 style, applied to the smoothed region): +40 boost at the edge pixel
```

(d) Combined output ≈ smoothed value + sharpening response = 30 + 40 = 70 (illustrative)

(e) The combined pipeline yields a result that is both noise-suppressed and edge-enhanced, which is generally visually superior to applying either filter alone.

29. Multi-Threshold Segmentation

(a) Multi-level segmentation is needed when an image contains more than two distinct intensity classes (e.g. background, mid-tone object, bright object), so a single binary threshold cannot separate all of them correctly.

(b) Thresholds at 50 and 100 are justified because they fall in the gaps between the given pixel clusters (20 | 60 | 120,180), cleanly separating the data into three natural classes.

(c) Apply segmentation to [20, 60, 120, 180] with thresholds (50, 100):

(d) Classes: 20 → Class 1 (<50); 60 → Class 2 (50–100); 120 → Class 3 (>100); 180 → Class 3 (>100)

(e) The four pixels split into three meaningful intensity classes, demonstrating how multiple thresholds allow finer-grained segmentation than a single global threshold.

30. Boundary Detection

(a) Boundary extraction identifies the closed contour separating a segmented region from its background/other regions, which is essential for shape analysis, object measurement, and recognition.

(b) The method is justified as a two-step pipeline: an edge detector (e.g. Sobel, Q8/Q19) locates candidate boundary pixels from local gradients, and edge linking (Q23) then joins these fragments into continuous, closed contours.

(c) Apply edge detection followed by linking:

```
1. Edge detection (e.g. Sobel) → candidate edge pixels, possibly with gaps (cf. Q8, Q19).
2. Edge linking (cf. Q23) → gaps such as [1,0,1,0,1] are closed to [1,1,1,1,1].
```

(d) Result: a continuous boundary/contour outlining the segmented region.

(e) The final linked contour traces the true object boundary, which can then be used for shape descriptors, area/perimeter measurement, or further region-based analysis.